ADMINSENA

Mapa técnico y funcional del proyecto

Arquitectura por dominios, modelo de datos, portales por rol, vistas, flujos funcionales, factories, seeders, pruebas y backlog técnico.

Presentación técnica · 24 de septiembre de 2026
PHP 8.1+ · Laravel 10 · Vite 6 · Tailwind CSS 4 · MySQL / SQLite

Objetivo: explicar cómo está construido AdminSena, qué muestra cada portal y cuál es la siguiente frontera de calidad.

1. Resumen ejecutivo

Una vista de alto nivel para presentar el alcance actual y la dirección técnica.

AdminSena es una plataforma Laravel para organizar la formación técnica del SENA. Integra un portal público, autenticación por rol, operación institucional, seguimiento docente, proceso del aprendiz y admisión de aspirantes. La organización por dominios permite que cada área de negocio tenga sus propios controladores, requests, modelos, rutas y vistas.

146	20	134	14	16 / 100
rutas	tablas de dominio	vistas Blade	módulos	tests / assertions

Lectura ejecutiva

La base funcional ya es considerable: Eloquent expresa las relaciones, los portales limitan consultas al usuario y la UI tiene un shell responsive común. La prioridad siguiente es cerrar autorización por objeto, publicación, privacidad de documentos y contratos inconsistentes.

Inventario	Cantidad	Lectura
Controladores de dominio	24	CRUD y flujos por módulo
Factories	20	Una por tabla/modelo, incluidas pivotes
Seeders	22	20 de dominio + raíz + compatibilidad
Migraciones	24	21 de dominio + 3 internas
Vistas	134	Dashboards, CRUD, layouts y componentes
Pruebas	5 archivos	Navegación, autorización y factories/seeders

2. Contenido

El documento se puede recorrer como una presentación por capítulos o como una referencia técnica.

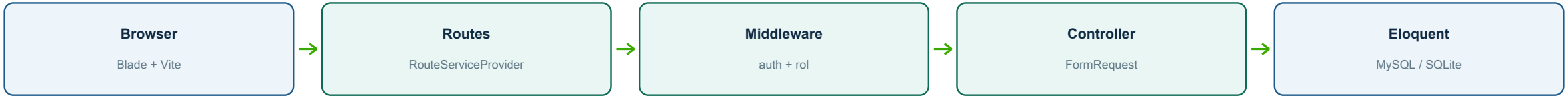
#	Capítulo	Contenido
01	Resumen ejecutivo	Alcance, cifras y conclusión
02	Arquitectura	Stack, routing, middleware y ciclo de una petición
03	Mapa de dominios	Los 14 módulos y sus responsabilidades
04	Modelo de datos	20 tablas y relaciones principales
05	Portales y vistas	Admin, Instructor, Aprendiz y Aspirante
06	Validación y negocio	Enums, fechas, ownership y estados
07	Factories y seeders	Datos de prueba y dependencias
08	Calidad y riesgos	Pruebas achieved y backlog priorizado
09	Roadmap y glosario	Próximas fases y términos clave

Cómo leer este documento

Las secciones 3 a 7 describen lo que existe. La sección 8 separa las funcionalidades logradas de los riesgos todavía observables en el código. La sección 9 propone una ruta de evolução sin presentar recomendaciones como si ya estuvieran implementadas.

3. Arquitectura

La aplicación separa infraestructura, dominio y presentación sin abandonar las convenciones de Laravel.



La base de datos es el límite de consistencia; Blade es la capa de presentación.

3.1 Componentes principales

Capa	Implementación	Responsabilidad
HTTP	Controllers + FormRequest	Reciben parámetros, validan y coordinan casos de uso
Dominio	app/Dominios	Agrupa modelos, rutas, requests y controladores por negocio
Persistencia	Eloquent + migrations	Relaciones, enums, FKs, timestamps y datos de prueba
Presentación	Blade + Tailwind	Dashboards, formularios, tablas, estados y navegación
Assets	Vite + Bootstrap Icons	Compilación de CSS/JS e iconografía
Pruebas	PHPUnit + SQLite	Aislamiento de la base y pruebas de relaciones/autorización

3.2 Routing y middleware

- RouteServiceProvider carga los archivos web/API de cada dominio explícitamente; por eso el proyecto no depende de un único archivo de rutas gigante.
- Los grupos web incluyen cookies, sesión, CSRF y bindings. El grupo api agrega throttling de 60 solicitudes por minuto.
- Los alias admin, instructor, aprendiz y aspirante aplican autorización básica por rol; algunos controladores añaden autorización por objeto y ámbito.
- Los archivos de API de los dominios están preparados como puntos de extensión, pero actualmente están vacíos; la API real es un trabajo futuro.

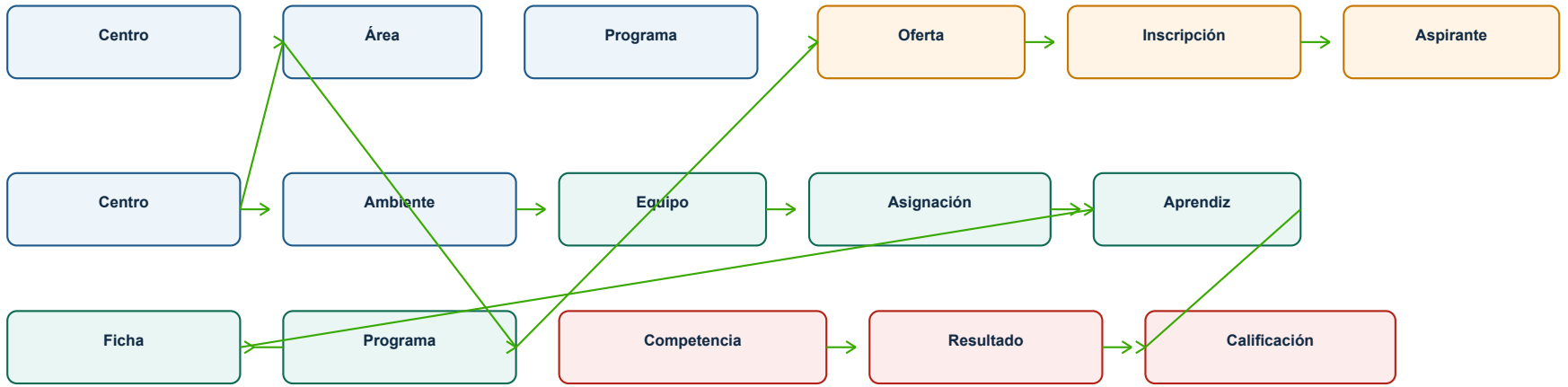
Decisión arquitectónica positiva

La separación por dominios mejora la navegabilidad del código y hace visible qué parte del sistema debe cambiar para una nueva regla de negocio. La siguiente mejora es convertir parte de las reglas de autorización en Policies reutilizables.

4. Mapa de dominios

Los 14 módulos pueden agruparse en identidad, estructura, formación, operación y comunicación.

Dominio	Responsabilidad	Entidad / flujo principal
Auth	Acceso y catálogo público	Portada, login, registro, noticias, ofertas
Usuarios	Identidad y perfiles de rol	User + Admin/Teacher/Apprentice/Aspirant
Centros	Sedes institucionales	Trainingcenter
Áreas	Organización académica	Area → Program
Programas	Oferta formativa base	Program → Offer
Ambientes	Espacios de formación	Environment → Equipment
Equipos	Inventario operativo	Equipment → Assignment
Fichas	Grupo de aprendices	Course_group + pivote docente
Competencias	Unidades de desempeño	Competition + pivote docente
Resultados	Resultados y calificaciones	Result + Grade
Asignaciones	Entrega de equipos	Assignment
Ofertas	Convocatorias	Offer
Inscripciones	Admisión y estados	Registration
Noticias	Comunicación pública	News



Las relaciones pivote agregan docentes a la ficha y a las competencias sin reemplazar el docente principal.

5. Identidad, acceso ydomains públicos

La experiencia pública y la autenticación son la puerta de entrada a los cuatro portales.

5.1 Auth y experiencia pública

Permite descubrir la institución, acceder al sistema y consultar noticias/ofertas sin sesión.

Modelos / tablas	Vistas principales
OfertasyNoticiasController, LoginController, RegisterController y el modelo User.	Auth/inicio.blade.php, Auth/login.blade.php, Auth/register.blade.php, Auth/noticias.blade.php, Auth/noticia.blade.php, Auth/ofertas.blade.php, Auth/oferta.blade.php.

Qué permite

- La portada comunica la propuesta institucional y los módulos principales. El acceso actual usa Auth::attempt, regeneración de sesión y redirección por rol.
- El catálogo de noticias filtra por título y categoría; el de ofertas busca por nombre de programa, código o área.
- Cada tarjeta enlaza a un show público con contenido completo, imagen, metadatos y fechas.
- Las páginas show devuelven 404 cuando el recurso no está publicado.

Reglas y límites

- Las ofertas requieren publicada = true; las noticias requieren publicada = true y estado = 'Publicado'.
- El login actual usa los roles capitalizados que espera el enum de users.
- El request de registro todavía acepta roles en minúsculas: es una inconsistencia que debe normalizarse.
- La portada contiene cifras de presentación como +120, +35 y 1,248; no son consultas a la base actual.

5.2 Usuarios y perfiles

Separa la identidad común del perfil especializado que cada rol necesita para operar.

Modelos / tablas	Vistas principales
User, Admin, Teacher, Apprentice, Aspirant y ConsultasController.	Usuarios/Admin/*, Usuarios/Instructores/*, Usuarios/Aprendices/*, Usuarios/Aspirantes/*.

Qué permite

- Admin gestiona su perfil, consulta usuarios por rol y ve el dashboard institucional.

- Instructor mantiene área, especialidad, fechas y tipo de instructor.
- Aprendiz mantiene ficha, estado y fechas formativas; el alcance de edición es un punto que requiere endurecimiento.
- Aspirante mantiene EPS y documentos de admisión.

Reglas y límites

- Cada perfil se enlaza con user_id y el usuario tiene relaciones hasOne.
- Las consultas de los portales usan el usuario autenticado como punto de entrada, no un ID tomado de la URL.

6. Dominios institucionales y catálogo

La estructura académica describe dónde se forman los grupos y con qué recursos.

6.1 Centros

Representa las sedes donde se dictan programas y se organizan ambientes.

Modelos / tablas	Vistas principales
Trainingcenter; relaciones hasMany hacia Area y Environment.	Centros/index.blade.php, create, edit, show y formulario parcial.

Qué permite

- CRUD de nombre/dirección.
- Listado ordered por fecha de creación.
- Detalle del centro.
- Formularios con mensajes de validación en español.

Reglas y límites

- El request exige nombre de mínimo 5 caracteres, letras/espacios y dirección de mínimo 10 caracteres.
- La ruta actual no tiene middleware auth/admin en su grupo web; es un riesgo de seguridad.

6.2 Áreas

Agrupa la oferta académica de un centro y conecta áreas con programas.

Modelos / tablas	Vistas principales
Area pertenece a Trainingcenter y tiene muchos Program.	Areas/index.blade.php, create, edit, show y estilos parciales.

Qué permite

- CRUD de áreas.
- Listado con centro y conteo de programas.
- Selección de centro en alta/edición.

Reglas y límites

- El nombre tiene mínimo 3 caracteres.
- La existencia del centro se valida en AreaRequest.

6.3 Programas

Define la formación académica que puede generar ofertas y grupos.

Modelos / tablas	Vistas principales
Program pertenece a Area y tiene muchas ofertas (offerts en el método de relación).	Programas/index.blade.php, create, edit, show y formulario parcial.

Qué permite

- CRUD de programas.
- Código único y nivel controlado por enum.
- Listado con área, centro y conteo de ofertas.

Reglas y límites

- Niveles permitidos: Tecnico y Tecnologo.
- La relación offerts es un nombre histórico; la tabla real es offers.

6.4 Ambientes y equipos

Administra espacios y recursos que luego se entregan a aprendices.

Modelos / tablas	Vistas principales
Environment pertenece a Trainingcenter; Equipment pertenece a Environment.	Ambientes/* y Equipos/* con listados, formularios y detalles.

Qué permite

- Ambientes: nombre, capacidad, tipo y centro.
- Equipos: nombre, marca, tipo y ambiente.
- Las relaciones permiten navegar del equipo a su ambiente y centro.

Reglas y límites

- Tipos de ambiente: Software, TICS, Laboratorio, Cocina.
- La capacidad mínima validada es 1; las factories usan valores realistas de 10–40.

7. Formación y operación académica

Aquí se conectan fichas, docentes, competencias, resultados, calificaciones y equipos.

7.1 Fichas de formación

Una ficha agrupa aprendices de un programa y asigna docentes.

Modelos / tablas	Vistas principales
Course_group, CourseGroupTeacher; Program y Teacher como dependencias.	Fichas/index.blade.php, create, edit, show y Fichas/form.blade.php.

Qué permite

- CRUD de grupos.
- Consulta de programa, instructor principal, instructores pivote y aprendices.
- El controlador sincroniza el docente pivote al crear/editar.

Reglas y límites

- Código único y capacidad positiva.
- El instructor principal debe ser de tipo Planta.
- Las consultas de instructor consideran teacher_id y la relación pivote.

7.2 Competencias

Representa competencias con periodo y responsables docentes.

Modelos / tablas	Vistas principales
Competition, CompetitionTeacher; relación directa a Teacher y N:M.	Competencias/index.blade.php, create, edit, show y form.blade.php.

Qué permite

- CRUD administrativo.
- Asignación de instructores mediante sync.
- Las fechas sostienen la agenda y el filtro de resultados.

Reglas y límites

- El request valida teacher_id, pero el controlador consume teachers[]; el contrato de formulario debe unificarse.
- El código no tiene una restricción unique de base aunque el request la aplica.

7.3 Resultados y calificaciones

Distingue el resultado de aprendizaje de la evaluación concreta de cada aprendiz.

Modelos / tablas	Vistas principales
Result pertenece a Competition; Grade pertenece a Result, Apprentice y opcionalmente Teacher.	Resultados/*, Resultados/Instructor/* y Resultados/Aprendiz/index.blade.php.

Qué permite

- Admin crea, edita, filtra y elimina resultados sin grades.
- Instructor abre una matriz por ficha/competencia y guarda con updateOrCreate.
- Aprendiz consulta únicamente sus grades y su resumen por estado.

Reglas y límites

- Estados: Aprobado, Reprobado, Pendiente.
- La combinación (result_id, apprentice_id) es única.
- El modelo Result ya no debe incluir apprentice_id; la tabla se reestructuró.

7.4 Asignaciones

Conecta un equipo físico con un aprendiz y su fecha de entrega/devolución.

Modelos / tablas	Vistas principales
Assignment pertenece a Equipment y Apprentice.	Asignaciones/* para administración/instructor y Asignaciones/Aprendiz/* para el aprendiz.

Qué permite

- Admin consulta todas.
- Instructor crea, edita, elimina y filtra dentro de sus fichas.
- Aprendiz lista y abre solo las suyas.
- El resumen distingue activas, finalizadas y devueltas.

Reglas y límites

- Estados: Activo, Inactivo.
- La devolución debe ser igual o posterior a la asignación.
- Debe corregirse la búsqueda del Aprendiz para usar whereHas('equipment', ...).

8. Admisión y comunicación

Ofertas, inscripciones y noticias conectan la plataforma con el público y la comunidad.

8.1 Ofertas

Representa convocatorias con programa, capacidad, calendario y publicación.

Modelos / tablas	Vistas principales
Offer pertenece a Admin y Program; tiene muchas Registration.	Ofertas/*, Auth/ofertas.blade.php, Auth/oferta.blade.php y vistas del Aspirante.

Qué permite

- Admin crea, edita, elimina, publica y despublica ofertas.
- El catálogo público busca por nombre de programa, código o área y cada tarjeta abre un show.
- El Aspirante solo ve ofertas publicadas y puede abrir el detalle para gestionar su inscripción.
- La capacidad y las fechas determinan si una oferta admite nuevas inscripciones.

Reglas y límites

- Secuencia de fechas obligatoria en el request.
- El listado y el show público solo cargan ofertas con publicada = true.
- El show muestra programa, centro, código, cupos, inscripciones activas y calendario.

8.2 Inscripciones

Gestiona el estado de un Aspirante dentro de una Oferta.

Modelos / tablas	Vistas principales
Registration pertenece a Aspirant y Offer.	Inscripciones/* para Admin y Inscripciones/Aspirante/* para autoservicio.

Qué permite

- Admin filtra por búsqueda, oferta y estado; puede cambiar el estado.
- Aspirante consulta, crea, edita y cancela con verificaciones de propiedad.

- El detalle carga oferta/programa y datos del aspirante.
- Las fechas generan una línea de tiempo.

Reglas y límites

- Solo Registrado permite edición.
- Solo Registrado/En proceso permite cancelación.
- Se evita una inscripción activa duplicada por lógica en el controller.
- Documentos requeridos en la primera inscripción.

8.3 Noticias

Administra publicaciones institucionales y las muestra en el portal público.

Modelos / tablas	Vistas principales
News pertenece a Trainingcenter y opcionalmente a Admin.	Noticias/*, Auth/noticias.blade.php, Auth/noticia.blade.php, formularios y cards públicas.

Qué permite

- CRUD administrativo.
- Búsqueda por título y categoría.
- Estados editoriales y bandera publicada.
- Cada noticia publicada tiene un show individual con imagen, categoría, fecha, centro y descripción completa.

Reglas y límites

- El listado y el show requieren publicada = true y estado = 'Publicado'.
- Las imágenes se almacenan bajo news/public.
- categoria y publicada deben mantenerse en el contrato del request y formulario administrativo.

9. Diccionario de datos

La siguiente tabla conecta cada tabla con su responsabilidad y dependencia principal.

Tabla	Responsabilidad	Claves/campos clave	Depende de
users	Identidad/autenticación	rol, email, password, identificación	Base de perfiles
trainingcenters	Sedes	nombre, dirección	Raíz institucional
admins	Perfil administrador	user_id, cargo, oficina	users
aspirants	Perfil aspirante	user_id, EPS, documentos	users
areas	Áreas	centro, nombre	trainingcenters
environments	Ambientes	centro, capacidad, tipo	trainingcenters
teachers	Perfil docente	user_id, área, especialidad, tipo	users, areas
equipments	Inventario	ambiente, name, brand, type	environments
programs	Programas	área, código, nivel	areas
news	Contenido	centro, admin, estado, categoría	trainingcenters, admins
course_groups	Fichas	programa, teacher, código, capacidad	programs, teachers
apprentices	Aprendices	user, ficha, estado, fechas	users, course_groups
offers	Ofertas	admin, programa, capacidad, fechas, publicada	admins, programs
competitions	Competencias	teacher, código, fechas	teachers
results	Resultados	competition, código, nombre	competitions
grades	Calificaciones	result, apprentice, teacher, estado	results, apprentices, teachers
registrations	Inscripciones	aspirant, offer, estado, fecha	aspirants, offers
assignments	Asignaciones	equipment, apprentice, fechas, estado	equipments, apprentices
competition_teacher	Pivote docente	competency_id, teacher_id	competitions, teachers
course_group_teacher	Pivote docente	course_group_id, teacher_id	course_groups, teachers

Nota sobre nomenclatura

competency_id conserva el nombre histórico aunque la tabla referenciada es competitions. Cambiarlo requiere una migración nueva, no editar una migración que ya pudo aplicarse en otra base.

10. Portales, vistas y navegación

El sistema usa un shell común y diferencia el contenido por el prefijo de ruta.

10.1 Layout y navegación

- Layouts/Usuarios/AppAdmin.blade.php es el layout activo y selecciona sidebar/header por request()->routeIs().
- Includes/Usuarios/Sidebar.blade.php recibe configuración de portal y dibuja navegación desktop + barra inferior móvil.
- Los sidebars de rol usan rutas nombradas, iconos SVG, estados activos y secciones plegables.
- Los componentes page-header, metric, empty-state y status-badge permiten consistencia visual.
- Tailwind aplica layouts fluidos, tablas con scroll horizontal, grids responsivos y estados vacíos.

10.2 Vista por rol

Portal	Rutas	Vistas/funciones	Alcance
Admin	/admin/* y CRUDs protegidos	Dashboard, estructura, oferta, noticias, inscripciones, resultados, usuarios	Totales + filtros + CRUD
Instructor	/instructor/*	Aprendices, fichas, resultados, asignaciones, actividades, agenda	Ámbito por ficha/docente
Aprendiz	/aprendiz/*	Dashboard, ficha, asignaciones, resultados, cronograma, perfil	Solo datos propios
Aspirante	/aspirante/*	Ofertas, inscripciones, resultados, fechas, perfil	Ofertas publicadas + ownership
Público	/ y /auth.*	Inicio, noticias, ofertas, login, registro	Catálogo público

10.3 Qué muestra cada dashboard

Dashboard	Métricas/visibles	Acciones rápidas
Admin	Centros y aprendices reales; algunos valores de presentación	Centros, áreas, ambientes, equipos
Instructor	Aprendices, fichas, competencias, asignaciones, progreso	Aprendices, fichas, resultados, asignaciones
Aprendiz	Ficha, equipos, resultados y progreso	Ficha, tareas, resultados, cronograma

Dashboard	Métricas/visibles	Acciones rápidas
Aspirante	Inscripciones, documentos, estado y avance	Perfil, inscripción, resultados, fechas

11. Validación y reglas de negocio

Los FormRequests defienden la entrada; los modelos y controllers defienen la transición y el alcance.

Área	Regla	Punto de control
Identidad	CC/TI/CE/PPT; email e identificación únicos; password hasheada	RegisterRequest / User model
Centro	Nombre 5–255; dirección mínimo 10	TrainingcentersRequest
Ambiente	Centro existente; capacidad ≥1; tipo enum	EnvironmentRequest
Programa	Área existente; código único; nivel enum	ProgramRequest
Ficha	Programa/docente; código único; capacidad ≥1; docente Planta	CourseGroupRequest
Oferta	Fechas ordenadas; capacidad positiva; imagen <=2MB	OfferRequest
Asignación	Equipo/aprendiz existentes; devolución >= asignación; estado enum	AssignmentRequest
Inscripción	Aspirante/oferta; documentos; ownership; estado editable/cancelable	RegistrationRequest / model
Calificación	IDs autorizados a ficha/competencia; estado enum; unique compuesto	GradeRequest / controller

11.1 ownership y estados

- El Instructor no puede abrir una ficha, competencia o asignación fuera de las relaciones que lo relacionan con la ficha/competencia.
- El Aprendiz solo puede abrir una asignación cuyo apprentice_id coincide con su perfil.
- El Aspirante solo puede ver/modificar inscripciones cuyo aspirant_id coincide con su perfil.
- Una inscripción cancelada deja de contar para el proceso activo y la capacidad de la oferta.
- Un grade solo se guarda si el resultado pertenece a la competencia autorizada y el aprendiz a la ficha autorizada.

Distinción importante

La autorización por rol no siempre equivale a autorización por objeto. Los controllers de asignaciones, resultados e inscripciones sí añaden filtros de pertenencia; la siguiente etapa debe extender ese patrón a todos los módulos sensibles.

12. Factories y seeders

La nueva capa de datos hace que el proyecto pueda probarse sin fixtures manuales dispersos.

12.1 Factories

Se crearon 20 factories en database/factories, con relaciones anidadas y estados válidos para los enums del esquema.

Grupo	Factories
Identidad	UserFactory, AdminFactory, TeacherFactory, ApprenticeFactory, AspirantFactory
Estructura	TrainingcenterFactory, AreaFactory, EnvironmentFactory, ProgramFactory, EquipmentFactory
Formación	CourseGroupFactory, CompetitionFactory, ResultFactory, GradeFactory
Operación/admisión	AssignmentFactory, OfferFactory, RegistrationFactory, NewsFactory
Pivotes	CompetitionTeacherFactory, CourseGroupTeacherFactory

- La resolución global en AppServiceProvider permite usar Model::factory() para modelos App\Dominios con factories planas.
- Las factories respetan la estructura final de results y grades, uniques, FKs, enums y fechas de oferta.
- Los estados permiten construir casos válidos: rol, aprobado/reprobado/pendiente, registro/cancelación, publicación y tipo docente.

12.2 Seeders

Los 20 seeders de dominio se cargan desde DatabaseSeeder. Cada uno resuelve sus dependencias y es idempotente mediante updateOrCreate o sync.

Grupo	Seeders
Identidad	UserSeeder, AdminSeeder
Estructura	TrainingCenterSeeder, AreaSeeder, EnvironmentSeeder, ProgramSeeder
Personas	TeacherSeeder, CourseGroupSeeder, ApprenticeSeeder, AspirantSeeder
Académico	CompetitionSeeder, ResultSeeder, GradeSeeder, CompetitionTeacherSeeder
Operación/admisión	EquipmentSeeder, AssignmentSeeder, OfferSeeder, RegistrationSeeder

Grupo	Seeders
Contenido	NewsSeeder, CourseGroupTeacherSeeder

Credenciales demo

Todos los usuarios generados por la factory y el seeder usan 12345678. Ejemplos: admin@adminsena.com, instructor1@adminsena.com, aprendiz1@adminsena.com y aspirante1@adminsena.com.

13. Calidad, riesgos y backlog

La suite valida la base funcional, pero todavía deben cerrarse algunos riesgos de autorización y publicación.

13.1 Evidencia de calidad

12	77	0	2×
tests	assertions	factories fallidas	seeder idempotente

Comprobación	Resultado	Observación
Smoke de factories	Correcto	Crea una instancia de cada modelo/tabla de dominio
DatabaseSeeder	Correcto	Carga la cadena de dependencias
Idempotencia	Correcto	Segunda ejecución conserva conteos
Migraciones SQLite	Correcto	Rama SQLite de restructuring y enum Cancelada verificadas
Suite PHPUnit	16 / 100	Ejecutada en SQLite en memoria
Pint / Vite	Correcto	Formato y build verificados

13.2 Riesgos priorizados

Prioridad	Área	Riesgo	Acción
P0	Auth	Authenticate usa route('login') inexistente	Corregir a auth.login y probar invitados
P0	Seguridad	CRUD de estructura sin auth/admin	Envolver grupos con middleware
P0	Registro	Roles minúsculos y perfiles no creados	Normalizar y crear perfil en transacción
P0	Binding	Resource con parámetro/modelo desalineado	Unificar {inscripcion} y el método
P0	Privacidad	Documentos en disco public	Disco privado + descarga autorizada
P1	Publicación	Reglas editoriales entre request, modelo y vista	Policy de publicación única

Prioridad	Área	Riesgo	Acción
P1	Negocio	Duplicados lógicos sin constraints DB	Uniques parciales/transacciones
P1	UX/API	Métricas hardcodeadas y API vacía	Métricas reales; contratos API

14. Roadmap recomendado

Una secuencia de entrega que prioriza seguridad antes de añadir nuevas capacidades.

Fase	Trabajo	Resultado esperado
Fase 1 · Seguridad	Authenticate, CRUDs, roles/onboarding, binding y documentos privados	Cerrar accesos indebidos y contratos rotos
Fase 2 · Dominio	Policies, publicación, filtros, uniques, métricas reales y auditoría	Datos confiables y consistentes
Fase 3 · Operación	Logout/remember, jobs de archivos, reportes, alertas y observabilidad	Operación mantenible
Fase 4 · Entrega	CI, pruebas navegador, backups, despliegue y documentación	Release reproducible

Conclusión

AdminSena ya tiene una arquitectura reconocible y datos de prueba confiables. El siguiente nivel no es maquillar más vistas: es hacer que cada dato, cada ruta y cada documento tenga un contrato de acceso y de publicación claro.

14.1 Comandos de referencia

Objetivo	Comando
Desarrollo	composer install && npm install && php artisan serve
Build frontend	npm run build
Suite	php artisan test --compact
Carga completa de demo	php artisan db:seed --class=DatabaseSeeder
Base nueva con demo	php artisan migrate:fresh --seed
Formato	vendor/bin/pint --test

15. Anexo de referencia

Rutas y archivos que conviene conocer para presented el proyecto.

15.1 Archivos clave

Área	Ubicación
Routing	app/Providers/RouteServiceProvider.php
Middleware	app/Http/Kernel.php + app/Http/Middleware/*
Modelos	app/Dominios/*/Models/*.php
Controladores	app/Dominios/*/Controllers/*.php
Validación	app/Dominios/*/Requests/*.php
Layout	resources/views/Layouts/Usuarios/AppAdmin.blade.php
Sidebar	resources/views/Includes/Usuarios/*/Sidebar.blade.php
Componentes	resources/views/components/portal/*.blade.php
Factories	database/factories/*.php
Seeders	database/seeders/*.php
Migraciones	database/migrations/*.php
Pruebas	tests/Feature/*.php

15.2 Glosario exprés

Término	Significado
Ficha	Course_group: grupo de aprendices vinculado a programa y docente
Competencia	Unidad de desempeño con periodo y responsables
Resultado	Result: resultado de aprendizaje de una competencia

Término	Significado
Grade	Calificación de un aprendiz para un resultado
Oferta	Convocatoria con programa, capacidad y calendario
Inscripción	Vínculo entre un Aspirante y una Oferta
Ámbito	Datos visibles según rol, ficha o propietario

Nota final

Este documento describe el estado observado del repositorio y no una certificación de producción. Antes de una entrega real, ejecutar pruebas HTTP sobre un entorno aislado, revisar la base migrada y cerrar los riesgos P0/P1 listados.